

Báo Cáo Review Public API Trên 9 Tiêu Chí

Nhóm 6

Thông tin chia việc:

- 1 + Tổng hợp báo cáo. Nguyễn Hải An
- 2 + 3. Bùi Hoàng Tùng
- 4 + 5. Lê Khánh Hưng
- 6 + 7. Trần Duy Thương
- 8 + 9. Lưu Việt Dũng

API: <https://discord.com/api>. Đây là một REST API giúp người dùng có thể tương tác được với dữ liệu phần mềm Discord trên hệ thống của họ.

1. Resources được đặt tên theo danh từ.

Khi đặt tên cho resource, một API nên: định nghĩa resources theo danh từ và không được dùng động từ cho chúng; các hành động thực hiện lên resources thì được định nghĩa qua các HTTP methods.

Đối với Discord API, các resources như: Application, Audit Log, Channel, Emoji, Message, Poll,... được phân tách cụ thể thành các endpoint riêng biệt với tên sử dụng chính các danh từ đó.

- **Resource 1: /users** | Gồm có các methods sau:

- GET /users/@me
- GET /users/{user.id}
- PATCH /users/@me
- GET /users/@me/guild
- GET /users/@me/guilds/{guild.id}/member
- DELETE /users/@me/guilds/{guild.id}
- POST /users/@me/channels
- POST /users/@me/channels
- GET /users/@me/connections
- GET /users/@me/applications/{application.id}/role-connection
- PUT /users/@me/applications/{application.id}/role-connection
- DELETE /users/@me/applications/{application.id}/role-connection

- **Resource 2: /emoji** | Gồm có các methods sau:

- GET /guilds/{guild.id}/emojis
- GET /guilds/{guild.id}/emojis/{emoji.id}
- POST /guilds/{guild.id}/emojis
- PATCH /guilds/{guild.id}/emojis/{emoji.id}

- DELETE /guilds/{guild.id}/emojis/{emoji.id}
- GET /applications/{application.id}/emojis
- GET /applications/{application.id}/emojis/{emoji.id}
- POST /applications/{application.id}/emojis
- PATCH /applications/{application.id}/emojis/{emoji.id}
- DELETE /applications/{application.id}/emojis/{emoji.id}

Discord đã sử dụng *users*, *applications*, *role-connection*, *channels*, *connections*, *guilds*, *member*, *emojis* là các danh từ chỉ resources.

Thông qua hai ví dụ, ta thấy được rằng các endpoints và resources được đảm bảo được đặt tên theo danh từ, được truy cập theo hệ thống phân cấp, và được thực hiện lên thông qua các methods chung.

2. Naming nhất quán.

Khi viết URI: toàn bộ path phải theo lowercase, các từ trong path được nối liền bởi ‘kebab-case’ hay ‘-’, các từ trong query được nối liền bởi ‘snake-case’ hay ‘_’, và các collections phải sử dụng số nhiều.

Đối với Discord API, họ đã tuân thủ một hệ thống quy tắc đặt tên (naming convention) cực kỳ chặt chẽ và đồng nhất xuyên suốt toàn bộ tài liệu API của họ.

A. Lowercase và kebab-case cho path:

- *Tiêu chuẩn:* Các URL path phải viết thường và dùng dấu gạch ngang (-) để nối các từ.
- *Thực tế Discord API:* Toàn bộ các endpoint của Discord đều được viết thường. Đối với các tài nguyên (resource) có tên gồm nhiều từ, Discord áp dụng triệt để kebab-case.
- *Ví dụ:* /guild-scheduled-events (Sự kiện đã lên lịch của máy chủ), /application-role-connection-metadata (Metadata kết nối role của ứng dụng), /stage-instances (Các phiên Stage).

B. Snake case cho query parameters và request/response body:

- *Tiêu chuẩn:* Các tham số truy vấn trên URL và các khóa (key) trong JSON payload phải dùng định dạng snake_case (chữ thường, nối bằng dấu gạch dưới).
- *Thực tế Discord API:* Discord không sử dụng camelCase hay PascalCase ở bất kỳ đâu trong giao tiếp HTTP. Toàn bộ dữ liệu truyền nhận đều chuẩn hóa theo snake_case.
- Ví dụ Query String: ?with_counts=true, ?max_age=86400, ?with_localizations=true. Hay trong JSON Body: Trong object User, các trường được đặt tên là avatar_decoration_data, mfa_enabled, public_flags.

C. Plurals (Danh từ số nhiều) cho Collection:

- *Tiêu chuẩn:* Các endpoint đại diện cho một tập hợp tài nguyên phải sử dụng danh từ số nhiều.
- *Thực tế Discord API:* Discord tuân thủ hoàn toàn nguyên tắc thiết kế tài nguyên RESTful này.
- Ví dụ về collections: /users (Tập hợp người dùng), /guilds (Tập hợp máy chủ), /channels (Tập hợp kênh). Và Ví dụ về document (Tài nguyên cụ thể): Khi muốn truy cập một tài nguyên cụ thể, Discord sử dụng ID nối tiếp collection: /users/{user.id} hoặc /channels/{channel.id}/messages/{message.id}.

Kết luận: Discord API đạt chuẩn tiêu chí. Hệ thống định danh nhất quán giúp lập trình viên dễ dàng đoán trước cấu trúc URL và payload mà không cần phải liên tục tra cứu tài liệu, giảm thiểu tối đa các lỗi sai chính tả do nhầm lẫn giữa các convention (camelCase vs snake_case).

3. Status Codes có nghĩa.

Tiêu chí này yêu cầu: mỗi response dùng status code đúng đắn và phù hợp; không trả trực tiếp 200 + Error ngay trong body.

Discord API đã sử dụng mã trạng thái HTTP (HTTP Status Codes) như một phần cốt lõi của giao thức giao tiếp, tận dụng tối đa ý nghĩa ngữ nghĩa (semantics) của từng mã code theo đúng chuẩn IETF.

A. Mỗi response dùng code phù hợp:

Discord ánh xạ cực kỳ chính xác hành động (HTTP Method) với kết quả trả về:

- **Nhóm thành công (2xx):**
 - 200 OK: Request lấy dữ liệu (GET), cập nhật (PATCH) diễn ra thành công.
 - 201 Created: Request tạo mới tài nguyên (POST) thành công. (Ví dụ: POST /channels/{channel.id}/messages trả về 201 kèm theo đối tượng message vừa tạo).
 - 204 No Content: Request xóa tài nguyên (DELETE) thành công. Lúc này, body trả về rỗng vì không cần thiết phải gửi thêm dữ liệu gì.
 -
- **Nhóm lỗi Client (4xx):**
 - 400 Bad Request: Định dạng request sai, thiếu tham số bắt buộc.
 - 401 Unauthorized: Lỗi xác thực (Thiếu token, token hết hạn hoặc sai định dạng).
 - 403 Forbidden: Request được xác thực nhưng user/bot bị thiếu Quyền (Permissions) để thực hiện hành động.

- 404 Not Found: Không tìm thấy tài nguyên (Ví dụ: ID của message hoặc channel đã bị xóa trước đó).
 - 405 Method Not Allowed: Gửi sai HTTP Method (Vd: gửi POST vào endpoint chỉ hỗ trợ GET).
 - 429 Too Many Requests: Vi phạm Rate Limit (Gửi quá nhiều request trong thời gian ngắn). Discord trả về kèm theo header Retry-After để báo cho client biết khi nào được gửi tiếp.
- **Nhóm lỗi Server (5xx):** Discord dùng 500 Internal Server Error hoặc 502 Bad Gateway để báo hiệu hệ thống của họ đang gặp sự cố.

B. Không trả 200 + error trong body:

- **Tiêu chuẩn:** Một antipattern phổ biến trong thiết kế API là mọi request đều trả về mã HTTP 200, sau đó client phải đọc JSON body (ví dụ: {"success": false, "error": "Lỗi gì đó"}) để biết thực sự thành công hay thất bại.
- **Thực tế Discord API:** Discord **tuyệt đối không** sử dụng antipattern này. Mọi lỗi nghiệp vụ hay lỗi hệ thống đều được phản hồi bằng đúng chuẩn HTTP Code đầu 4xx hoặc 5xx. Mã 200/201/204 chỉ xuất hiện khi request thực sự thành công. Khi có lỗi (như đã phân tích ở Tiêu chí 5), Discord sẽ trả về HTTP Status Code (ví dụ: 400 Bad Request) đi kèm với cấu trúc JSON mô tả chi tiết nguyên nhân lỗi.

Kết luận: Discord API đạt tiêu chí này. Việc tuân thủ nghiêm ngặt HTTP Status Codes mang lại lợi ích to lớn cho việc xử lý ngoại lệ (Exception Handling). Các HTTP Client (như Axios, Node-Fetch, hay Requests trong Python) sẽ tự động ném ra Exception khi nhận mã 4xx/5xx, giúp Developer tách biệt rõ ràng giữa luồng xử lý dữ liệu thành công (Try) và luồng xử lý lỗi (Catch).

4. Idempotency rõ ràng.

API cần phải: có Idempotency-Key cho POST, các phương thức PUT/DELETE thì phải idempotent. Các trường hợp idempotency thì cần có tài liệu hóa rõ ràng.

A. Kiểm tra POST có Idempotency-Key:

Khi thực hiện 2 lệnh POST trùng lặp, HTTP trả 201 cho lần đầu và 200 cho các lần sau. Tuy nhiên, response trả về không có Idempotency-Key, và dù có thêm Idempotency-Key thủ công ở đầu header thì vẫn cho kết quả HTTP 201 lần đầu và 200 cho các lần sau:

```
C:\Users\Hung>curl -i -X POST "https://discord.com/api/v10/applications/1556625277847863359/commands" -H "Authorization: Bot MTU1NjYyNTI3Nzg5Mzg2MzZlOQ.Gw0lht.erbklDp7Z7haQ-uRZMG0qaxZXEjvK11X6IQxd8" -H "Content-Type: application/json" -H "Idempotency-Key: 12345678-test-idempotency" -d '{"name":"id-test1","description":"Test POST idempotency","type":1}'
HTTP/1.1 201 Created
```

```
C:\Users\Hung>curl -i -X POST "https://discord.com/api/v10/applications/1556625277847863359/commands" -H "Authorization: Bot MTU1NjYyNTI3Nzg8Nzg2MzML0Q.Gwolt.erbklDp7Z7haQ-uRZWGQqaxZXEjvK11X6IQxd8" -H "Content-Type: application/json" -H "Idempotency-Key: 12345678-test-idempotency" -d '{"name":"id-test","description":"Test POST idempotency","type":1}'
HTTP/1.1 200 OK
```

Vì vậy, Discord API không có tính Idempotency với lệnh POST

B. Kiểm tra Idempotency của PUT:

Khi thực hiện 2 lệnh PUT liên tiếp với cú pháp tương đồng nhau, HTTP đều trả về 200. Tuy nhiên, khi sử dụng GET để truy xuất dữ liệu thì chỉ ra 1 kết quả, vì vậy có thể PUT lần 2 đã ghi đè trực tiếp lần 1 thay vì trả về lỗi. Kết quả 2 lần PUT đã cho thấy:

```
C:\Users\Hung>curl -i -X PUT "https://discord.com/api/v10/applications/1556625277847863359/commands" -H "Authorization: Bot MTU1NjYyNTI3Nzg8Nzg2MzML0Q.Gwolt.erbklDp7Z7haQ-uRZWGQqaxZXEjvK11X6IQxd8" -H "Content-Type: application/json" -d '{"name":"put-test","description":"PUT id-put-test","type":1}'
HTTP/1.1 200 OK
```

Kết quả GET:

```
C:\Users\Hung>curl -s "https://discord.com/api/v10/applications/1556625277847863359/commands" -H "Authorization: Bot MTU1NjYyNTI3Nzg8Nzg2MzML0Q.Gwolt.erbklDp7Z7haQ-uRZWGQqaxZXEjvK11X6IQxd8"
[{"id":"1556666145157677217","application_id":"1556625277847863359","version":"1556666145157677218","default_member_permissions":null,"type":1,"name":"put-test","description":"PUT id-put-test","dm_permission":true,"contexts":null,"integration_types":[0,1],"nsfw":false}]
```

Vì 2 lần PUT không tạo 2 bản ghi trùng lặp nên có thể coi Discord API có tính Idempotency với lệnh PUT.

C. Kiểm tra Idempotency của DELETE:

Khi thực hiện liên tiếp 2 lệnh xóa cùng 1 đối tượng message trong cùng 1 kênh của server, lần 1 sẽ xóa thành công và không trả nội dung, còn lần 2 trả về response báo lỗi của riêng Discord API.

```
C:\Users\Hung>curl -X DELETE "https://discord.com/api/v10/channels/1556624442396778377/messages/1556626763440726088" -H "Authorization: Bot MTU1NjYyNTI3Nzg8Nzg2MzML0Q.GUE1AA.3JzERmT3-TtOoeY-knvlJp1ShX8RLOeu_QnXmc"

C:\Users\Hung>curl -X DELETE "https://discord.com/api/v10/channels/1556624442396778377/messages/1556626763440726088" -H "Authorization: Bot MTU1NjYyNTI3Nzg8Nzg2MzML0Q.GUE1AA.3JzERmT3-TtOoeY-knvlJp1ShX8RLOeu_QnXmc"
{"message": "Unknown Message", "code": 10008}
```

Thử với tin nhắn khác, chi tiết lần 1 cho thấy:

```
C:\Users\Hung>curl -i -X DELETE "https://discord.com/api/v10/channels/1556624442396778377/messages/1556624686436244521" -H "Authorization: Bot MTU1NjYyNTI3Nzg8Nzg2MzML0Q.GUE1AA.3JzERmT3-TtOoeY-knvlJp1ShX8RLOeu_QnXmc"
HTTP/1.1 204 No Content
```

Lần 2 cho thấy:

```
C:\Users\Hung>curl -i -X DELETE "https://discord.com/api/v10/channels/1556624442396778377/messages/1556624688436244521"
-H "Authorization: Bot HTU1NjYyNTI3Nzg8Nzg2HzM10Q.GUE1AA.3JzERmT3-Tt0oeY-knwLJpiShX8Rl0eu_QnXmc"
HTTP/1.1 404 Not Found
```

Vì vậy Discord API có tính Idempotency với lệnh DELETE.

D. Tài liệu hóa:

Trong link về API Reference của Discord API, mục Consistency có ghi rằng “Clients should operate on events and results from the API in as much of an idempotent behavior as possible.”, có nghĩa là khuyến cáo người dùng áp dụng tính Idempotent khi tự sử dụng ứng dụng. Ngoài ra không có tài liệu cụ thể về Idempotency cho từng method, vì vậy có thể coi Discord API không có tài liệu cụ thể về tính Idempotency.

Kết luận: Discord API chỉ tuân thủ tính Idempotency theo cùng cách hoạt động ở một số method, không phải tất cả. Vì vậy Discord API chưa thực sự có tính Idempotency rõ.

5. Error response có cấu trúc (dựa trên RFC 7807).

API khi trả về error response phải theo chuẩn RFC 7807/9457 - chuẩn hóa định dạng Problem Details for HTTP APIs với ‘Content-Type’ là ‘application/problem+json’ kèm theo 5 trường chính: type, title, instance, detail, status

Discord API không sử dụng cấu trúc trả lỗi RFC 7807 mà sử dụng cấu trúc trả về lỗi được thiết kế riêng.

Cấu trúc 1 response lỗi mẫu của Discord API:

```
{
  "code": 50035,
  "errors": {
    "access_token": {
      "_errors": [
        {
          "code": "BASE_TYPE_REQUIRED",
          "message": "This field is required"
        }
      ]
    }
  },
  "message": "Invalid Form Body"
}
```

Trong đó:

“code”: mã lỗi của Discord.

“errors”: thông tin chi tiết của lỗi hoặc thành phần lỗi.

“message”: thông báo tổng quát về lỗi.

và bên trong error có thể có thêm các trường mã lỗi và thông báo về lỗi.

Khi đối chiếu với RFC 7087:

A. “type”:

- RFC 7807 sử dụng trường “type” để trả về URI định danh loại lỗi, qua đó người dùng xác định được lỗi nào đang xảy ra.
- Discord API không có trường nào trả về trực tiếp URI định danh loại lỗi như RFC.

B. “title”:

- RFC 7807 sử dụng “title” để tóm tắt ngắn nội dung lỗi
- Discord API không có trường “title” mà có trường “message” với mục đích tương tự. Tuy nhiên “message” lại tồn tại như biến thể của “detail” trong body của “errors”, khiến cho “message” và “title” không tương đồng nhau.

C. “status”:

- RFC 7807 sử dụng “status” để trả về status code theo format của HTTP, mục đích đối chiếu với mã HTTP được trả về cùng với response và cho người dùng biết lỗi gì.
- Discord API không có trường nào tương tự “status” để trả về mã lỗi HTTP trong response mà chỉ dùng mã được trả kèm response. Mục “code” cũng trả về mã lỗi nhưng mã này không theo mẫu status code của HTTP.

D. “detail”:

- RFC 7807 sử dụng “detail” để cho biết chi tiết occurrence cụ thể của lỗi là gì
- Discord API có phần body riêng trong “errors” cũng có chức năng tương tự là mô tả occurrence hiện tại ở message, tuy nhiên đây là cơ chế riêng và không theo đúng cấu trúc của phần detail trong RFC 7807.

E. “instance”:

- RFC 7807 có thể dùng “instance” để chỉ cụ thể phần nào đang gây lỗi trong request của người dùng.
- Discord API không có trường nào tương tự để trả về vị trí lỗi của người dùng trong request.

Kiểm tra Content-Type trả về:

- Đối với RFC 7807, Content-Type được trả về khi gặp lỗi là application/problem+json, để biểu diễn chi tiết lỗi bằng JSON.
- Discord API cũng sử dụng JSON cho response lỗi, nhưng vẫn trả kiểu application/json khi gặp lỗi.

Kết luận: Discord API khi trả response lỗi không trả về các trường được quy định theo tiêu chuẩn RFC 7807 và cũng không phải Content-Type chuẩn RFC 7807 là application/problem+json. Thay vào đó, Discord API sử dụng format tự thiết kế riêng. Vì vậy, Discord API không tuân thủ RFC 7807 về cấu trúc Problem Details.

6. Pagination

API khi sử dụng collections nên phải: có kế hoạch phân trang cho từng danh sách (cursor/offset/keyset/...) và hỗ trợ limit.

Đối với các collections trong Discord API, phân trang chỉ được sử dụng trong một số collections duy nhất.

A. Collection có Pagination:

- Messages: GET /channels/{channel.id}/messages — Có pagination thông qua các tham số limit, before, after, around.
- Guild Members: GET /guilds/{guild.id}/members — Có pagination thông qua các tham số limit, after.

B. Collection không có Pagination:

- Guild Channels: GET /guilds/{guild.id}/channels — Không có tham số pagination
- Guild Roles: GET /guilds/{guild.id}/roles — Không có tham số pagination
- Guild Invites: GET /guilds/{guild.id}/invites — Không có tham số pagination

Kết luận: Discord API hỗ trợ pagination ở một số collection như Messages và Guild Members, nhưng không áp dụng cơ chế pagination cho tất cả collection.

7. Filter/Sort đa dạng

A. Collection có Filter/Sort

Messages Search: GET /guilds/{guild.id}/messages/search — Hỗ trợ lọc theo nội dung, người gửi (author_id), kênh (channel_id), trạng thái ghim (pinned) và các điều kiện khác. Hỗ trợ sắp xếp theo thời gian (timestamp) hoặc mức độ liên quan (relevance) thông qua sort_by, kết hợp với sort_order khi phù hợp.

B. Collection không có Filter/Sort đa dạng được công bố

Guild Channels: GET /guilds/{guild.id}/channels — Trả về danh sách kênh của server, không công bố các tham số lọc và sắp xếp tùy chỉnh như filter, sort_by.

Guild Roles: GET /guilds/{guild.id}/roles — Trả về danh sách role của server, không công bố cơ chế lọc và sắp xếp tùy chỉnh.

Sparse Fieldsets: Chưa có bằng chứng cho thấy Discord cung cấp cơ chế chung để client tùy chọn các trường dữ liệu trả về bằng tham số fields.

Kết luận: Discord hỗ trợ Filter/Sort đa dạng tại một số endpoint, tiêu biểu là tìm kiếm tin nhắn. Tuy nhiên, các khả năng này không được áp dụng thống nhất cho tất cả collection; đồng thời, chưa có đủ bằng chứng để khẳng định Discord hỗ trợ sparse fieldsets trên toàn bộ API.

8. Authentication & Security

API phải có: Token bảo mật phải được truyền thông qua HTTP Header (ví dụ: Authorization: Bearer <token>), tuyệt đối không gửi qua URL query string (vì URL token sẽ bị ghi lại ở server access log, browser history, header Referer...). API đồng thời phải phản hồi thông tin Rate Limit rõ ràng và có thông điệp hướng dẫn khi bị vượt quá ngưỡng.

Đối với Discord API:

A. Cơ chế Xác thực (Authentication):

- Discord API bắt buộc xác thực qua HTTP Header với định dạng:
 - Dành cho Bot: Authorization: Bot <token>
 - Dành cho OAuth2 (User Access Token): Authorization: Bearer <token>
- Discord không cho phép và nghiêm cấm việc truyền token thông qua URL query parameter (ví dụ: ?token=...).
- *Đánh giá:* Đạt chuẩn rất tốt so với nguyên tắc thiết kế bảo mật của slide.

B. Cơ chế Kiểm soát Tốc độ (Rate Limiting):

- Discord áp dụng cơ chế Rate Limit rất chi tiết (per-route, global, gateway).
- Server gửi kèm các HTTP response header để client chủ động theo dõi:
 - X-RateLimit-Limit: Số lượng request tối đa.
 - X-RateLimit-Remaining: Số request còn lại trong window.
 - X-RateLimit-Reset: Thời gian reset limit (Epoch seconds).
 - X-RateLimit-Bucket: Định danh cho bucket chứa route đó.
- Khi client bị giới hạn (vượt quá rate limit), Discord trả về mã lỗi 429 Too Many Requests kèm header Retry-After (tính theo giây/mili giây) và response body chi tiết chứa retry_after.
- *Đánh giá:* Hoàn toàn khớp với chuẩn HTTP Status Code 429 và Retry-After header trong slide bài giảng.

Kết luận: Thiếu Idempotency Key cho các tác vụ quan trọng (Write Actions): Mặc dù bảo mật xác thực và rate-limit rất tốt, Discord API thiếu cơ chế Idempotency-Key header cho các phương thức POST hoặc PATCH tạo/thực hiện giao dịch (ví dụ: gửi tin nhắn, mua quà tặng/nitro, mua Sticker). Khi gặp sự cố chập chờn mạng làm request retry tự động, việc thiếu Idempotency Key khiến bot hoặc ứng dụng dễ gặp tình trạng gửi trùng tin nhắn nhiều lần hoặc thực hiện hành động dư thừa trên hệ thống.

9. Versioning & Deprecation

API phải tích hợp Version Prefix (tiền tố phiên bản) trực tiếp vào URL ngay từ thiết kế ban đầu (ví dụ: /api/v1/...); có chính sách, tài liệu hướng dẫn chuyển đổi và thông báo ngừng hỗ trợ phiên bản cũ rõ ràng để tránh breaking change cho client.

Đối với Discord API:

A. Versioning (Đánh số phiên bản):

- Discord triển khai phiên bản công khai dưới dạng Path Versioning:
 - Base URL: https://discord.com/api/v{version} (Ví dụ hiện tại là https://discord.com/api/v10).
- Các API request không truyền version mặc định sẽ chuyển về API version mặc định (Discord khuyến khích luôn khai báo rõ version trong URL).
- *Đánh giá:* Đạt chuẩn. Việc đặt version prefix /v10 ngay trên path giúp client dễ dàng xác định phiên bản đang làm việc.

B. Deprecation (Ngừng hỗ trợ):

- Discord duy trì tài liệu công khai lịch sử danh sách các phiên bản (API Versions status):
 - *Available:* Phiên bản khả dụng.
 - *Deprecated:* Phiên bản không khuyến khích sử dụng, ngừng cập nhật tính năng mới.
 - *Discontinued:* Phiên bản đã đóng hoàn toàn (Client gọi vào sẽ nhận lỗi 400 hoặc 404).
- Discord đưa ra mốc thời gian chuyển giao khá rõ ràng cho lập trình viên khi nâng cấp từ v6 v8 v9 v10.
- *Đánh giá:* Đạt chuẩn. Giúp phát triển ứng dụng bền vững, tránh tình trạng ứng dụng của bên thứ ba bị dừng đột ngột (Client breakage).

Kết luận: Khả năng thương lượng phiên bản linh hoạt (Content Negotiation / Header Versioning) và Deprecation Header còn hạn chế:

- Discord chỉ hỗ trợ duy nhất Path Versioning (/api/v10/users). Thiếu sự hỗ trợ cho Header-based Versioning (ví dụ: Accept-Version hoặc X-API-Version), điều này ép buộc lập trình viên phải thay đổi cứng URL ở mọi endpoint khi nâng cấp phiên bản.
- Khi một endpoint hoặc một field cụ thể chuẩn bị bị loại bỏ (Deprecate), Discord chủ yếu đưa thông tin lên trang Documentation thay vì trả về HTTP Header Sunset hoặc Deprecation chuẩn RFC trực tiếp trong API response. Điều này khiến lập trình viên khó tự động hóa việc bắt cảnh báo ứng dụng đang dùng tính năng sắp bị khai tử thông qua log của ứng dụng.